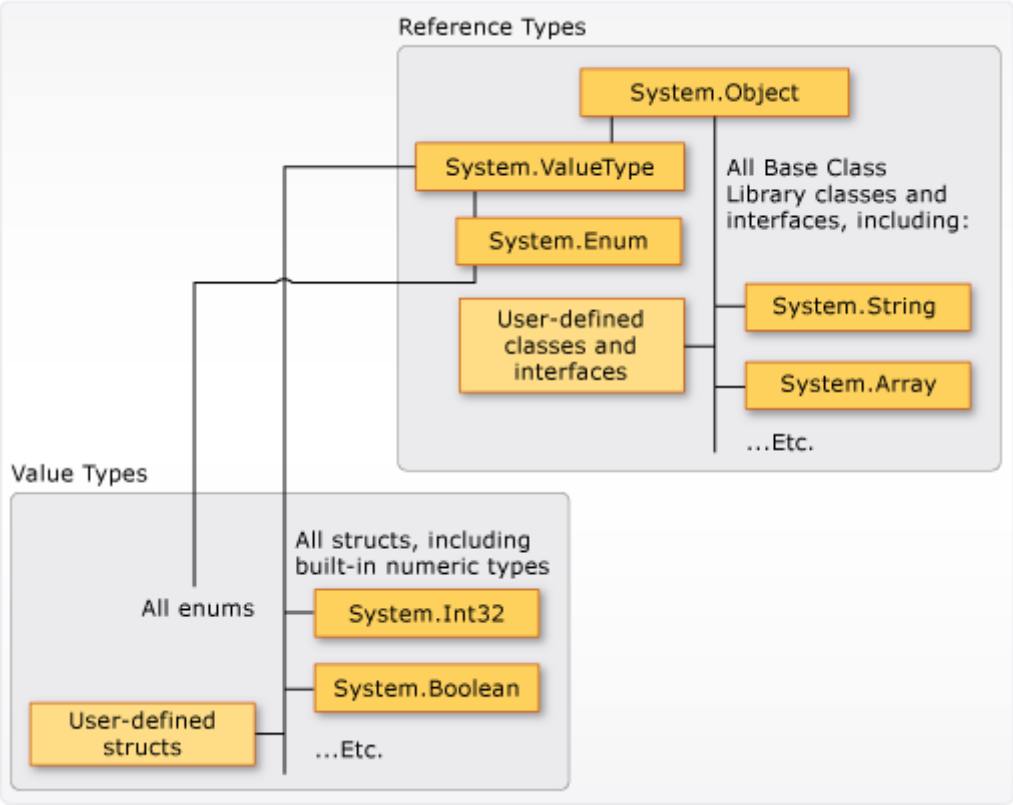
C#基础（1）

1 类型系统



- C#中有两种类型，值类型和引用类型
- 值类型有两种，enum和struct（包括用户定义struct）
- enum、struct、valuetype的继承关系见图

1.1 值类型

1.1.1 内置数据类型

每种值类型都有一个默认构造函数来初始化该类型默认值，声明一个值为0的整型的等价写法：

```
int i = new int();
Int32 i = new Int32();
int i = 0;
Int32 i = 0;
```

内置数据类型的范围和默认值：

类型	描述	范围	默认值
bool	布尔值	True 或 False	False
byte	8 位无符号整数	0 到 255	0
char	16 位 Unicode 字符	U +0000 到 U +ffff	'\0'

类型	描述	范围	默认值
decimal	128 位精确的十进制值，28-29 有效位数	(-7.9 x 1028 到 7.9 x 1028) / 100 到 28	0.0M
double	64 位双精度浮点型	(+/-)5.0 x 10-324 到 (+/-)1.7 x 10308	0.0D
float	32 位单精度浮点型	-3.4 x 1038 到 + 3.4 x 1038	0.0F
int	32 位有符号整数类型	-2,147,483,648 到 2,147,483,647	0
long	64 位有符号整数类型	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807	0L
sbyte	8 位有符号整数类型	-128 到 127	0
short	16 位有符号整数类型	-32,768 到 32,767	0
uint	32 位无符号整数类型	0 到 4,294,967,295	0
ulong	64 位无符号整数类型	0 到 18,446,744,073,709,551,615	0
ushort	16 位无符号整数类型	0 到 65,535	0

1.2 引用类型

定义：引用变量时不存储变量实际数据，只包含对变量的引用

1.3 值类型与引用类型的区别

1. 二者在内存中存放位置不同

   示例1：

```
int num = 100;
int[] nums = { 1, 2, 3, 4, 5 };
Console.WriteLine(num);
Console.WriteLine(nums);
```

   运行结果：

```
100
System.Int32[] // 输出引用
```

   示例2：

```
class TestClass
{
    public int x;
    public static string y;
}

void Test1()
{
```

```
    var a=1;
    var b=new TestClass();
    var c=a;
    var d=b;
    var e=d.x;
    var f=TestClass.y;
}
```

- a是一个值类型，数据直接保存在栈上
- b是一个引用类型实例，部署在托管堆上，并保存一个引用在栈上
  - 引用类型TestClass的字段x、y都部署在托管堆上
- c和a是一样的值类型，赋值操作相当于创建了一个a的副本，且a与c互不影响
- d和b是一样的引用类型，赋值操作相当于将b的引用地址赋值给d，并指向堆中的实例对象
- e和d.x是一样的值类型，赋值操作相当于创建了一个d.x的副本，且e与d.x互不影响
- f和类中静态字段y是一样的引用类型，赋值操作相当于在内存中创建了一个对y的引用，并指向堆中y所在位置
- 值类型变量赋值时直接复制包含的值，在栈中开辟新的空间保存该值，并且与原变量互不影响；引用类型变量赋值时只复制变量的引用，原变量值改变时会同步

示例1：

```
class MyClass
{
    public int x = 0;
}

class Program
{
    static void Main(string[] args)
    {
        MyClass c1 = new MyClass();
        MyClass c2 = c1; // 引用
        int a = c1.x;
        c1.x = 1;
        Console.WriteLine(a);
        Console.WriteLine(c1.x);
        Console.WriteLine(c2.x);
    }
}
```

运行结果：

```
0 // 类中字段x的值改变不影响a
1 // 类中字段x的值改变会影响所有引用对象
1
```

string是一个特殊的引用类型，需要注意，示例2：

```
string a = "hello";
string b = a; // 引用
a = "world";
Console.WriteLine(a);
Console.WriteLine(b);
```

运行结果：

```
world
hello // a的值改变依旧不影响b，这是由于字符串的不可变性
```

2. 数组元素不管是值类型或引用类型，都存储在托管堆上
3. 类型嵌套：引用类型在栈中存储一个引用，实际存储位置在托管堆上（引用类型部署在托管堆上）；值类型作为字段时跟随所属实例存储，作为局部变量时存储在栈上

```
public class MyClass1
{
    private int _value1;
    public MyClass1()
    {
        _value1 = 0;
    }
    public void Method()
    {
        int _value2 = 0;
    }
}

public struct MyStruct
{
    private object _object1;
    public void Method()
    {
        _object1 = new object();
        object _object2 = new object();
    }
}

public class MyClass2
{
    private  MyClass1 _classValue1 = new MyClass1();
}

class Program
{
    static void Main(string[] args)
    {
        MyClass1 mc1 = new MyClass1();
        mc1.Method();
        MyStruct ms = new MyStruct();
        ms2.Method();
        MyClass2 mc2 = new MyClass2();
    }
```

```
}
```

- mc1是一个引用类型实例，因此部署在托管堆上，并被栈上一个引用所持有
- _value1是mc1的一个值类型字段，应跟随mc1部署在托管堆上
- _value2是值类型局部变量，因此部署在栈上
- ms是一个值类型实例，因此部署在栈上
- _object1是ms的一个引用类型字段，不存在跟随问题，故部署在托管堆上，并被一个引用所持有（该引用是ms的一部分）
- _object2是是引用类型局部变量，因此部署在托管堆上，并被栈上一个引用所持有
- mc2是一个引用类型实例，因此部署在托管堆上，并被栈上一个引用所持有，mc2._classValue1是一个引用类型实例，因此也部署在托管堆上，其引用则存储在mc2所在的托管堆区

4. 值类型内存管理效率高，不支持多态，适合做存储数据的载体；引用类型支持多态，适合用于定义应用程序的行为
5. 值类型都是密封的，不能派生出新的类型；引用类型可以
6. 值类型不能包含null值；引用类型可以

```
string a = null;
Console.WriteLine(a);
```

2 string

2.1 字符串初始化

初始化字符串的几种方式：

```
string a = "123";
string b = @"\\123"; // 强制不转义
System.String c = "123";
var d = "123";
char[] letters = { '1', '2', '3' };
string e = new string(letters); // 利用字符数组初始化字符串

```

运行结果：

```
123
\\123
123
123
123
```

2.2 字符串的不可变性

字符串对象创建后无法更改，看似对字符串的修改和操作实际都为其创建了新的字符串。

示例1：

```
string a = "hello ";
string b = "world";
a += b; // 实际创建了一个新的字符串"hello world"作为新的对象分配给a，程序运行结束后a的原始对象被释放
Console.WriteLine(a);
```

运行结果：

```
hello world
```

示例2：

```
string a = "hello";
string b = a; // 创建了对a的引用，指向a的原始对象
a = "world"; // 实际为a分配新的对象，a指向新的对象，由于存在引用，a的原始对象不被释放
Console.WriteLine(a);
Console.WriteLine(b); // b仍然指向a的原始对象
```

运行结果：

```
world
hello
```

2.3 字符串常用方法

```
string a = "abc";
Console.WriteLine(a.Length); // 字符串长度
Console.WriteLine(a.ToUpper()); // 大写
Console.WriteLine(a.ToLower()); // 小写
Console.WriteLine(a.Equals("ABC", StringComparison.OrdinalIgnoreCase)); // 比较字符串，忽略大小写

string b = "2023-7-27";
string[] date = b.Split(new char[] { '-' }); // 字符串分割
Console.WriteLine("{0}年{1}月{2}日", date[0], date[1], date[2]); // 字符串复合格式输出
Console.WriteLine(b.Substring(0, 4)); // 字符串截取
Console.WriteLine(b.IndexOf("23", 0)); // 从前往后寻找子串起始位置
Console.WriteLine(b.LastIndexOf('7', b.Length - 1)); // 从后往前寻找子串最后出现的位置
Console.WriteLine(b.StartsWith("19")); // 判断字符串打头
Console.WriteLine(b.EndsWith("27")); // 判断字符串结尾
if (b.Contains("27")) Console.WriteLine(b.Replace("27", "28")); // 判断字符串是否包含某个子串以及子串替换

string c = "   abc   ";
Console.WriteLine(c.TrimStart()); // 去掉字符串前面空格
Console.WriteLine(c.TrimEnd()); // 去掉字符串结尾空格
Console.WriteLine(c.Trim()); // 去掉字符串前后空格
Console.WriteLine(string.IsNullOrEmpty(c)); // 判断字符串是否为空
Console.WriteLine(string.Join("-", a, b, c)); // 用指定分隔符串联字符串
```

运行结果：

```
3
ABC
abc
True
2023年7月27日
2023
2
8
False
True
2023-7-28
abc
   abc
abc
False
abc-2023-7-27-   abc
```

2.4 stringBuilder

使用stringBuilder声明字符串：

```
System.Text.StringBuilder s = new System.Text.StringBuilder("abc");
s[0] = 'A';
Console.WriteLine(s);
```

运行结果：

```
Abc
```

用stringBuilder的好处：

1. StringBuilder类创建了字符串缓冲区以适应字符串的扩展，用于在程序执行多个字符串操作时提升性能，如ocr识别后对字符串快速处理。
2. 支持重新分配各个字符，可直接更改字符串内容而无需创建新字符串

适用场景：

1. 预期代码对字符串进行大量更改
2. 预期代码没有太多搜索操作，如IndexOf、StartsWith（StringBuilder不提供）

3 动态类型

作用：dynamic定义的变量和对其成员的引用绕过编译时类型检查，改为运行时进行。编译时，dynamic类型变量会编译为object类型变量，因此dynamic类型变量只在编译时存在，运行时不存在。

3.1 特性

1. 声明一个动态变量后，对其进行任何操作都不会引发编译错误：

```
dynamic x = 10;
x = x + "Hello"; // x 现在是一个字符串
x = x.ToUpper(); // x 仍然是一个字符串
x = x * 8; // x现在仍然是int
```

2. var与dynamic区别在于，var允许编写时省略变量类型，让编译器自动推断，推断出来后就会像对普通变量一样进行类型检查：

```
var x = 10; // x is an int
x = x + "Hello"; // 错误,无法将 string 类型转换为 int 类型
x = x.ToUpper();  // 错误 int 没有 ToUpper 方法
x = x * 8; // 正确,int可以执行 * 运算
```

3. dynamic类型保存值类型会发生装箱，这是由于dynamic本质也是引用类型，保存的是对object的引用，将值类型转换为object类型会发生装箱：

```
int x = 10; // 值类型
dynamic y = x; // 装箱操作
```

3.2 用途

1. 与其它动态语言（Python、Java）进行互操作

   使用dynamic调用python代码中定义的函数：

```
var engine = Python.CreateEngine(); // 创建Python引擎和作用域
var scope = engine.CreateScope();
// 执行Python代码来定义函数
engine.Execute(@"
def add(x, y):
    return x + y
", scope);
dynamic add = scope.GetVariable("add"); // 获取动态对象的函数
dynamic result = add(10, 20); //使用动态参数调用函数并获得动态结果
Console.WriteLine(result); // 输出30
```

2. 与COM（允许不同应用程序之间交换数据和功能）对象进行互操作

   使用dynamic创建一个Excel工作簿并添加一些数据

```
dynamic excelApp = new Excel.Application(); // 用动态类型的方式创建excel对象
excelApp.Visible = true; // 设置可见性为true
dynamic workbook = excelApp.Workbooks.Add(); // 作为动态类型添加excel工作簿
dynamic activeSheet = excelApp.ActiveSheet; // 获取激活的表用作对象
dynamic range = activeSheet.Range["A1", "C3"]; // 获取作为动态对象的单元格范围
range.Value = new object[3, 3] { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
// 将范围的值设置为对象数组
```

3.3 局限性

1. 动态类型会损失编译器提供的一些功能和保证，如智能感知、重构、代码分析、错误检测等
2. 动态类型会降低代码的可读性和可维护性，因为它们隐藏了变量的真实类型和行为
3. 动态类型会增加运行时异常的风险，如果它们找不到合适的成员或参数不匹配
4. 动态类型会影响代码的性能和效率，因为它们需要额外的步骤来解析和执行操作

4 类型转换

4.1 隐式转换

- 自动进行的类型转换，当目标数据类型可以容纳源数据类型的值时，C#编译器会自动完成转换，而不需要显式指定转换操作。
- 将一个较小范围的数据类型转换为较大范围的数据类型。
- 由于这种转换始终会成功且不会导致数据丢失，因此无需使用任何特殊语法。
- 对内置数据类型，若要存储的值不需截断或四舍五入即可适应变量（不丢失数据），则不需特殊语法就能直接转换，如int->long，float->double。

4.1.1 隐式数值转换

支持的隐式转换：

From	到
sbyte	`short`、`int`、`long`、`float`、`double`、`decimal` 或 `nint`。
byte	`short`、`ushort`、`int`、`uint`、`long`、`ulong`、`float`、`double`、`decimal`、`nint` 或 `nuint`
short	`int`、`long`、`float`、`double`、`decimal` 或 `nint`
ushort	`int`、`uint`、`long`、`ulong`、`float`、`double`、`decimal`、`nint` 或 `nuint`
int	`long`、`float`、`double`、`decimal` 或 `nint`
uint	`long`、`ulong`、`float`、`double`、`decimal` 或 `nuint`
long	`float`、`double` 或 `decimal`
ulong	`float`、`double` 或 `decimal`
float	`double`
nint	`long`、`float`、`double` 或 `decimal`
nuint	`ulong`、`float`、`double` 或 `decimal`

- 所有整型都可以隐式转换为浮点型，这是由于浮点型是用IEEE 754表示，表示范围更大，正数最大 $2^{127} * (2 - 2^{-23})$

从 int、uint、long、ulong、nint 或 nuint 到 float 的隐式转换以及从 long、ulong、nint 或 nuint 到 double 的隐式转换可能会丢失精准率，但绝不会丢失一个数量级。

```
int a = 100;
double d = a; //将int类型转换为double类型

float f = 3.14f;
double e = f; //将float类型转换为double类型
```

运行结果：

```
100
3.140000104904175
```

4.1.2 隐式引用转换

- 从派生类到基类的转换：

```
Derived d = new Derived();
Base b = d;
```

- 从类（实现了接口）到接口的转换
- 从任何数组类型到System.Array的转换
- 从任何代表类型到System.Delegate的转换
- 从空类型(null)到任何引用类型的转换

4.1.3 隐式枚举转换

隐式枚举转换允许把十进制整数0转换成任何枚举类型，对应其它的整数则不存在这种隐式转换，这是因为0始终是有效的枚举类型。

```
class Test
{
    enum Weekday
    {
        Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
    };
    public static void Main()
    {
        Weekday day;
        day = 0;
        Console.WriteLine(day);
    }
}
```

运行结果：

```
Sunday
```

如果枚举不是从0开始，会自动生成一个0的枚举类型：

```
class Test
{
    enum Weekday
    {
        Sunday = 1, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
    };
    public static void Main()
    {
        Weekday day;
        day = 0;
        Console.WriteLine(day);
    }
}
```

运行结果：

```
0
```

4.2 显式（强制）转换

- 显式类型转换，即强制类型转换。
- 显式转换是指将一个较大范围的数据类型转换为较小范围的数据类型
- 需要用户明确指定转换类型，需要强制转换运算符
- 强制转换会造成数据丢失

4.2.1 显式数值转换

强制转换 double 为 int，示例：

```
using System;

namespace TypeConversionApplication
{
    class ExplicitConversion
    {
        static void Main(string[] args)
        {
            double d = 5673.74;
            int i;

            // 强制转换 double 为 int
            i = (int)d;
            Console.WriteLine(i);
            Console.ReadKey();

        }
    }
}
```

运行结果：

```
5673
```

对于从一种整型到另一种整型的转换，编译器将针对转换进行溢出检测，如果没有发生溢出，转换成功，否则抛出一个OverflowException异常，这种检测还与编译器中是否设定了checked选项有关

```
long l = Int32.MaxValue;
l++;
int i = (int)l;
Console.WriteLine(i);
```

运行结果：

```
-2147483648
```

- 对于从float,double,或decimal到整型的转换，源变量的值通过舍入到最接近的整型值作为转换的结果。如果这个整型值超出了目标类型的值域，则将抛出一个OverflowException异常
- 对于从double到float的转换，double值通过舍入取最接近的float值。如果这个值太小，结果将变成正0或负0；如果这个值太大，将变成正无穷或负无穷。如果原double值是Nan，则转换结果也是NaN
- 对于从float或double到decimal的转换，源值将转换成小数形式并通过舍入取到小数点后28位（如果有必要的话）。如果源值太小，则结果为0；如果太大以致不能用小数表示，或是无穷和NaN,则将抛出InvalidCastException异常

4.2.2　显式引用转换

- 从基类到派生类的转换
- 从类（没有实现接口）到接口的转换
- 从System.Array到数组类型
- 从System.Delegate到代表类型

4.2.3　显式枚举转换

支持从sbye,byte,short,ushort,int,uint,long,ulong,char,float,double,或decimal到任何枚举类型，从任何枚举类型到、sbyte,byte,short,ushort,int,uint,long,ulong,char,float,double,或decimal，从任何枚举类型到任何其它枚举类型。实际上是枚举的元素类型和要转换的数据类型之间的转换。

从枚举类型转为int类型：

```
using System;

namespace 类型转换
{
    public enum QQState
    {
        OnLine,
        OffLine,
        Leave,
        Busy,
        QMe
    }
    class Program
    {
        static void Main(string[] args)
        {
```

```
            QQState state = QQState.OnLine;
            int n = (int)state;
            Console.WriteLine(n);
            Console.WriteLine((int)QQState.OffLine);
            Console.WriteLine((int)QQState.Leave);
            Console.WriteLine((int)QQState.Busy);
            Console.WriteLine((int)QQState.QMe);
            Console.ReadKey();
        }
    }
}
```

运行结果:

```
0
1
2
3
4
```

从int类型转为枚举类型:

```
using System;

namespace 类型转换
{
    public enum QQState
    {
        OnLine,
        OffLine,
        Leave,
        Busy,
        QMe
    }
    class Program
    {
        static void Main(string[] args)
        {
            int n1 = 0;
            QQState state = (QQState)n1;
            Console.WriteLine(state);
            Console.ReadKey();
        }
    }
}
```

运行结果:

```
OnLine
```

4.2.4 显式转换方法

4.2.4.1 cover类方法

类的 Convert 静态方法主要用于支持在 .NET 中与基数据类型进行转换。

支持的基类型为 Boolean、Char、SByte、Byte、UInt16、Int64、UInt32、Int16、UInt64、Int32、Single、Double、 Decimal、DateTime 和 String 。

```
using System;

class Program
{
    static void Main()
    {
        // Convert string to int
        string strNumber = "42";
        int intValue = Convert.ToInt32(strNumber);
        Console.WriteLine("intValue: " + intValue);

        // Convert string to double
        string strDouble = "3.14";
        double doubleValue = Convert.ToDouble(strDouble);
        Console.WriteLine("doubleValue: " + doubleValue);

        // Convert string to bool
        string strBool = "True";
        bool boolValue = Convert.ToBoolean(strBool);
        Console.WriteLine("boolValue: " + boolValue);

        // Convert int to string
        int number = 123;
        string strValue = Convert.ToString(number);
        Console.WriteLine("strValue: " + strValue);

        // Convert string to DateTime
        string strDate = "2023-07-31";
        DateTime dateValue = Convert.ToDateTime(strDate);
        Console.WriteLine("dateValue: " + dateValue);

        // Convert string to long
        string strLong = "1234567890";
        long longValue = Convert.ToInt64(strLong);
        Console.WriteLine("longValue: " + longValue);

        // Convert string to float
        string strFloat = "2.718";
        float floatValue = Convert.ToSingle(strFloat);
        Console.WriteLine("floatValue: " + floatValue);

        // Convert string to decimal
        string strDecimal = "123.456";
        decimal decimalValue = Convert.ToDecimal(strDecimal);
        Console.WriteLine("decimalValue: " + decimalValue);
    }
}
```

```


```

运行结果：

```
intValue: 42
doubleValue: 3.14
boolValue: True
strValue: 123
dateValue: 2023/7/31 0:00:00
longValue: 1234567890
floatValue: 2.718
decimalValue: 123.456
```

4.2.4.2 Parse方法

Parse()方法用于将任意字符串转换成任意类型，实际上是一种对字符串的解析。 要求等式左右两边的类型要匹配。

```
using System;

class Program
{
    static void Main()
    {
        // 使用 int.Parse() 将字符串转换为整数类型
        string intStr = "123";
        int intValue = int.Parse(intStr);
        Console.WriteLine("Parsed int value: " + intValue);

        // 使用 float.Parse() 将字符串转换为单精度浮点类型
        string floatStr = "3.14";
        float floatValue = float.Parse(floatStr);
        Console.WriteLine("Parsed float value: " + floatValue);

        // 使用 double.Parse() 将字符串转换为双精度浮点类型
        string doubleStr = "2.718281828459045";
        double doubleValue = double.Parse(doubleStr);
        Console.WriteLine("Parsed double value: " + doubleValue);

        // 使用 decimal.Parse() 将字符串转换为精准数据类型
        string decimalStr = "123456789.123456789";
        decimal decimalValue = decimal.Parse(decimalStr);
        Console.WriteLine("Parsed decimal value: " + decimalValue);

        // 使用 char.Parse() 将字符串转换为字符类型
        string charStr = "A";
        char charValue = char.Parse(charStr);
        Console.WriteLine("Parsed char value: " + charValue);
    }
}
```

输出：

```
Parsed int value: 123
Parsed float value: 3.14
Parsed double value: 2.718281828459045
Parsed decimal value: 123456789.123456789
Parsed char value: A
```

使用"TryParse"方法来进行转换，并处理转换错误的情况，使用 `int.TryParse` 进行整数转换的示例：

```
using System;

class Program
{
    static void Main()
    {
        string numberString = "42";
        int number;

        if (int.TryParse(numberString, out number))
        {
            Console.WriteLine($"Parsed integer: {number}");
        }
        else
        {
            Console.WriteLine("Invalid integer format");
        }
    }
}

```

输出：

```
Parsed integer: 42
```

当使用 `TryParse` 方法进行数据类型转换时，还可以结合条件运算符（三元运算符）来更简洁地处理转换结果，示例

```
using System;

class Program
{
    static void Main()
    {
        string numberString = "42";
        int number;

        string result = int.TryParse(numberString, out number)
            ? $"Parsed integer: {number}"
            : "Invalid integer format";

        Console.WriteLine(result);
    }
}
```

输出：

```
Parsed integer: 42
```

4.3 装箱与拆箱

4.3.1 装箱操作

- 装箱是值类型到 object 类型或到此值类型所实现的任何接口类型的隐式转换。
- 对值类型装箱会在堆中分配一个对象实例，并将该值复制到新的对象中。

将整型变量 i 进行装箱并分配给对象 o：

```
namespace 数据类型转换1
{
    internal class Program
    {
        static void Main(string[] args)
        {

            int i = 123;
           //将整型变量 i 进行了装箱并分配给对象o
            object o = i;
            Console.WriteLine(o);

        }
    }
}
```

- 此语句的结果是在堆栈上创建对象引用 o，而在堆上则引用 int 类型的值。
- 该值是赋给变量 i 的值类型值的一个副本。
- 以下装箱转换图说明了 i 和 o 这两个变量之间的差异：

示例：

```
internal class TestBoxing
{
    static void Main()
    {
        int i = 123;

        // 装箱操作
        object o = i;

        // 改变i值
        i = 456;

        System.Console.WriteLine("The value-type value = {0}", i);
        System.Console.WriteLine("The object-type value = {0}", o);
    }
}

```

运行结果：

```
The value-type value = 456
The object-type value = 123
```

4.3.2 拆箱（取消装箱）

- 拆箱是从 object 类型到值类型或从接口类型到实现该接口的值类型的显式转换。
- 消装箱操作包括：
- 检查对象实例，以确保它是给定值类型的装箱值。
- 将该值从实例复制到值类型变量中

示例：

```
int i = 123; // a value type
object o = i; // 装箱
int j = (int)o; // 拆箱
```

要在运行时成功取消装箱值类型，被取消装箱的项必须是对一个对象的引用，该对象是先前通过装箱该值类型的实例创建的。错误拆箱示例：

```
class TestUnboxing
    {
        static void Main()
        {
            int i = 123;
            object o = i;  // 装箱
            int j = (short)o;  //拆箱
            Console.WriteLine(j);
        }
    }
```

返回值：

```
Unhandled exception. System.InvalidCastException:
Unable to cast object of type 'System.Int32' to type 'System.Int16'.


##如何修改才可以正常运行：
    int j = (short)o;  //拆箱
修改为：
    int j = (int)o;  //拆箱
```

4.3.3 拆/装箱对性能的影响

- 装箱和取消装箱过程需要进行大量的计算。
- 对值类型进行装箱时，必须创建一个全新的对象。 这可能比简单的引用赋值用时最多长 20 倍。
- 取消装箱的过程所需时间可达赋值操作的四倍。
- 如果值类型必须被频繁装箱，那么在这些情况下最好避免使用值类型（例如在诸如 System.Collections.ArrayList 的非泛型集合类中）。

- 可通过使用泛型集合（例如 System.Collections.Generic.List）来避免装箱值类型。

5 变量和常量

5.1 变量

5.1.1 变量

变量（Variables）：用于存储和表示数据的名字，它可以在程序执行过程中改变其值。

```
int age; // 声明一个整数类型的变量age
age = 25; // 初始化age的值为25
age = 30; // 修改age的值为30
```

5.1.2 静态变量与实例变量

静态字段（Static Fields）：

- 静态字段属于类本身，而不是类的实例。在类的所有实例之间共享相同的静态字段。
- 静态字段在类加载时初始化，并在整个应用程序的生命周期内保持其值。
- 静态字段可以用来存储所有实例之间共享的数据。

```
class MyClass
{
    public static int Count; // 静态字段
}

// 访问静态字段
MyClass.Count = 10;
int count = MyClass.Count;
```

对比：

1. 静态变量使用`static`修饰，实例变量不用。若不显式初始化，静态变量和类中的实例变量都将被初始化为默认值（此外，数组元素也会被自动初始化为默认值）
2. 程序运行时静态变量就会被分配空间，而实例变量需要创建了实例对象才会被分配空间
3. 一个类的所有实例的同一静态变量都是同一个值，即静态成员只被创建一次，同时一个对象将其值改变，在其它对象上也会同步。实例变量属对象私有，不影响其它对象

```
class Test
{
    public static int a = 1;
    public int b = 1;
}
class Program
{
    static void Main(string[] args)
    {
        Test t1 = new Test();
        Test t2 = new Test();
        Test.a += 1;
        t1.b += 2;
        t2.b += 2;
```

```
        Console.WriteLine(Test.a);
        Console.WriteLine(t1.b);
        Console.WriteLine(t2.b);
    }
}
```

运行结果：

```
2
3
3
```

5.2 （静态）常量

常量（Constants）：具有恒定值的标识符，其值在整个程序执行过程中都保持不变。
特点：

1. 使用`const`修饰，在程序生命周期内不改变
2. 内置类型可声明为`const`，用户定义类型不可以
3. 常量的值对于某一个类的所有实例都是相同的，**需要像访问静态成员那样去访问const定义的常量**

与`readonly`对比：

	const	readonly
定义	声明的同时要设置常量值	声明的时候可以不需要进行设置常量值，可以在类的构造函数中进行设置（相当于类的成员）
类型限制	首先类型必须属于值类型范围（字符串除外），且其值不能通过new来进行设置	没有限制，可以用它定义任何类型的常量
对于类对象而言	对于所有类的对象而言，常量的值是一样的	对于类的不同对象而言，常量的值可以是不一样的
内存消耗	无	要分配内存，保存常量实体
综述	性能要略高，无内存开销，但是限制颇多，不灵活	灵活，方便，但是性能略低，且有内存开销

6 运算符

算数运算符：

运算符	描述	实例
+	把两个操作数相加	A + B 将得到 30
-	从第一个操作数中减去第二个操作数	A - B 将得到 -10

*	把两个操作数相乘	A * B 将得到 200
/	分子除以分母	B / A 将得到 2
%	取模运算符，整除后的余数	B % A 将得到 0
++	自增运算符，整数值增加 1	A++ 将得到 11
--	自减运算符，整数值减少 1	A-- 将得到 9

关系运算符：

运算符	描述	实例
==	检查两个操作数的值是否相等，如果相等则条件为真。	(A == B) 不为真。
!=	检查两个操作数的值是否相等，如果不相等则条件为真。	(A != B) 为真。
>	检查左操作数的值是否大于右操作数的值，如果是则条件为真。	(A > B) 不为真。
<	检查左操作数的值是否小于右操作数的值，如果是则条件为真。	(A < B) 为真。
>=	检查左操作数的值是否大于或等于右操作数的值，如果是则条件为真。	(A >= B) 不为真。
<=	检查左操作数的值是否小于或等于右操作数的值，如果是则条件为真。	(A <= B) 为真。

逻辑运算符：

运算符	描述	实例
&&	称为逻辑与运算符。如果两个操作数都非零，则条件为真。	(A && B) 为假。
||	称为逻辑或运算符。如果两个操作数中有任意一个非零，则条件为真。	(A || B) 为真。
!	称为逻辑非运算符。用来逆转操作数的逻辑状态。如果条件为真则逻辑非运算符将使其为假。	!(A && B) 为真。

位运算符：

运算符	描述	实例

&	如果同时存在于两个操作数中，二进制 AND 运算符复制一位到结果中。	(A & B) 将得到 12，即为 0000 1100
|	如果存在于任一操作数中，二进制 OR 运算符复制一位到结果中。	(A | B) 将得到 61，即为 0011 1101
^	如果存在于其中一个操作数中但不同时存在于两个操作数中，二进制异或运算符复制一位到结果中。	(A ^ B) 将得到 49，即为 0011 0001
~	按位取反运算符是一元运算符，具有"翻转"位效果，即0变成1，1变成0，包括符号位。	(~A) 将得到 -61，即为 1100 0011，一个有符号二进制数的补码形式。
<<	二进制左移运算符。左操作数的值向左移动右操作数指定的位数。	A << 2 将得到 240，即为 1111 0000
>>	二进制右移运算符。左操作数的值向右移动右操作数指定的位数。	A >> 2 将得到 15，即为 0000 1111

赋值运算符：

运算符	描述	实例
=	简单的赋值运算符，把右边操作数的值赋给左边操作数	C = A + B 将把 A + B 的值赋给 C
+=	加且赋值运算符，把右边操作数加上左边操作数的结果赋值给左边操作数	C += A 相当于 C = C + A
-=	减且赋值运算符，把左边操作数减去右边操作数的结果赋值给左边操作数	C -= A 相当于 C = C - A
*=	乘且赋值运算符，把右边操作数乘以左边操作数的结果赋值给左边操作数	C *= A 相当于 C = C * A
/=	除且赋值运算符，把左边操作数除以右边操作数的结果赋值给左边操作数	C /= A 相当于 C = C / A
%=	求模且赋值运算符，求两个操作数的模赋值给左边操作数	C %= A 相当于 C = C % A
<<=	左移且赋值运算符	C <<= 2 等同于 C = C << 2
>>=	右移且赋值运算符	C >>= 2 等同于 C = C >> 2
&=	按位与且赋值运算符	C &= 2 等同于 C = C & 2
^=	按位异或且赋值运算符	C ^= 2 等同于 C = C ^ 2
|=	按位或且赋值运算符	C |= 2 等同于 C = C | 2

杂项运算符：

运算符	描述	实例
sizeof()	返回数据类型的大小。	sizeof(int)，将返回 4.
typeof()	返回 class 的类型。	typeof(StreamReader);
&	返回变量的地址。	&a; 将得到变量的实际地址。
*	变量的指针。	*a; 将指向一个变量。
? :	条件表达式	如果条件为真 ? 则为 X : 否则为 Y
is	判断对象是否为某一类型。	If(Ford is Car) // 检查 Ford 是否是 Car 类的一个对象。
as	强制转换，即使转换失败也不会抛出异常。	Object obj = new StringReader("Hello"); StringReader r = obj as StringReader;

- `sizeof` 运算符返回给定类型的变量所占用的字节数，并且需要不安全上下文

```
sizeof(byte) // 1
sizeof(bool) // 1
sizeof(char) // 2
sizeof(short) // 2
sizeof(int) // 4
sizeof(float) // 4
sizeof(long) // 8
sizeof(double) // 8
```

运算符优先级

类别	运算符	结合性
后缀	() [] -> . ++ - -	从左到右
一元	+ - ! ~ ++ - - (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR	|	从左到右

逻辑与 AND	&&	从左到右
逻辑或 OR	||	从左到右
条件	?:	从右到左
赋值	= += -= *= /= %=>>= <<= &= ^= |=	从右到左
逗号	,	从左到右

7 条件分支

7.1 if else

7.1.1 if 语句

- 一个 if 语句 由一个布尔表达式后跟一个或多个语句组成。

```
val b = 0;
if((int)2.3 == 2){
    b = 4;
}
Console.WriteLine(b);
```

output

```
4
```

7.1.2 if...else 语句

- 一个 if 语句 后可跟一个可选的 else 语句，else 语句在布尔表达式为假时执行。

```
val b = 0;
if((int)2.3 == 3){
    b = 4;
}
else {
    b = 8;
}
Console.WriteLine(b);

```

output

```
8
```

7.1.3 嵌套 if 语句

- 在 C# 中，嵌套 if-else 语句是合法的，这意味着您可以在一个 if 或 else if 语句内使用另一个 if 或 else if 语句。

```
val b = 0;
if((int)2.3 == 2){
    b = 4;
    if(true)
        b = 8;
}
Console.WriteLine(b);
```

output

```
8
```

7.2 switch 语句

- 一个 switch 语句允许测试一个变量等于多个值时的情况。每个值称为一个 case，且被测试的变量会对每个 switch case 进行检查。
- 当被测试的变量等于 case 中的常量时，case 后跟的语句将被执行，直到遇到 break 语句为止。
- C# 不允许从一个 case 部分继续执行到下一个 case 部分。如果 case 语句中有已经执行，则必须包含 break 或其他跳转语句。
- 一个 switch 语句可以有一个可选的 default 语句，在 switch 的结尾。default 语句用于在上面所有 case 都不为 true 时执行的一个任务。default 也需要包含 break 语句，这是一个良好的习惯。
  无default

```
  int day = 4;
  switch (day)
  {
    case 1:
      Console.WriteLine("Monday");
      break;
    case 2:
      Console.WriteLine("Tuesday");
      break;
    case 3:
      Console.WriteLine("Wednesday");
      break;
    case 4:
      Console.WriteLine("Thursday");
      break;
    case 5:
      Console.WriteLine("Friday");
      break;
    case 6:
      Console.WriteLine("Saturday");
      break;
    case 7:
      Console.WriteLine("Sunday");
      break;
```

```
}
```

output

```
Thursday
```

有default

```
int day = 4;
switch (day)
{
  case 1:
    Console.WriteLine("Monday");
    break;
  case 2:
    Console.WriteLine("Tuesday");
    break;
  case 3:
    Console.WriteLine("Wednesday");
    break;
  case 4:
  case 5:
    Console.WriteLine("Friday");
    break;
  case 6:
    Console.WriteLine("Saturday");
    break;
  case 7:
    Console.WriteLine("Sunday");
    break;
  default:
    Console.WriteLine("unknow day");
    break;
}

Console.WriteLine("day = {0}",day);
```

output

```
Friday
day = 4
```

7.3 三目运算

- 三目运算（condition ? code1 : code2 ）：条件成立执行code1，否则code2

```
int a = 99;
var b = a == 100 ? 5050 : a;
Console.WriteLine(" b = {0}", b);
```

output

```
b = 99
```